

Adaptive FIR Filters

System identification through LMS algorithm or SGD learning algorithm in PyTorch

Give me some python code to demonstrate an adaptive filter using pytorch

ChatGPT Medium:

A clean way to demonstrate an **adaptive FIR filter in PyTorch** is to use PyTorch's automatic differentiation to adapt the FIR coefficients one sample at a time. Conceptually, this is very close to LMS: the filter observes an input $x[n]$, compares its output $y[n]$ with a desired signal $d[n]$, and adjusts its coefficients to minimize the squared error.

Here is a complete runnable example. It uses an unknown FIR system

$$h = [0.5, -0.3, 0.2, 0.1]$$

and asks the PyTorch adaptive filter to learn those coefficients.

```
import numpy as np
import torch
import matplotlib.pyplot as plt

# -----
# 1. Reproducibility
# -----

np.random.seed(1)
torch.manual_seed(1)

# -----
# 2. Generate an input signal
# -----

N = 3000

x = np.random.randn(N)

# -----
# 3. Define the unknown system we want to identify
# -----

h_true = np.array([0.5, -0.3, 0.2, 0.1])

M = len(h_true)
```

```

# Desired signal = output of unknown system
d = np.convolve(x, h_true, mode="full")[:N]

# Add a little measurement noise
noise_std = 0.05
d += noise_std * np.random.randn(N)

# Convert to PyTorch tensors
x_t = torch.tensor(x, dtype=torch.float32)
d_t = torch.tensor(d, dtype=torch.float32)

# -----
# 4. Define the adaptive FIR coefficients
# -----

# Start with all coefficients equal to zero
w = torch.nn.Parameter(torch.zeros(M))

# SGD plays the role of the adaptive coefficient update
learning_rate = 0.02

optimizer = torch.optim.SGD([w], lr=learning_rate)

# Storage for plotting
errors = []
weight_history = []

# -----
# 5. Adaptive filtering loop
# -----

for n in range(M - 1, N):

    # Input vector:
    #
    # [x[n], x[n-1], ..., x[n-M+1]]
    #
    x_vec = torch.flip(x_t[n-M+1:n+1], dims=[0])

    # Adaptive FIR filter output
    y = torch.dot(w, x_vec)

    # Error signal
    e = d_t[n] - y

    # Mean squared error for this sample
    loss = e**2

```

```

# Clear previous gradients
optimizer.zero_grad()

# Compute gradient
loss.backward()

# Update coefficients
optimizer.step()

errors.append(e.detach().item())
weight_history.append(w.detach().numpy().copy())

weight_history = np.array(weight_history)

# -----
# 6. Display results
# -----

print("True filter coefficients:")
print(h_true)

print("\nLearned filter coefficients:")
print(w.detach().numpy())

# -----
# 7. Plot coefficient convergence
# -----

plt.figure(figsize=(10, 5))

for k in range(M):
    plt.plot(weight_history[:, k],
             label=f"Adaptive w[{k}]")

    plt.axhline(h_true[k],
               linestyle="--",
               linewidth=1)

plt.xlabel("Iteration")
plt.ylabel("Coefficient value")
plt.title("Adaptive FIR Filter Coefficient Convergence")
plt.grid(True)
plt.legend()
plt.show()

# -----
# 8. Plot error signal

```

```

# -----

plt.figure(figsize=(10, 4))

plt.plot(errors)

plt.xlabel("Iteration")
plt.ylabel("Error")
plt.title("Adaptive Filter Error")
plt.grid(True)
plt.show()

# -----
# 9. Plot smoothed squared error
# -----

squared_error = np.array(errors)**2

window = 100
smoothed_error = np.convolve(
    squared_error,
    np.ones(window) / window,
    mode="valid"
)

plt.figure(figsize=(10, 4))

plt.plot(smoothed_error)

plt.xlabel("Iteration")
plt.ylabel("Average Squared Error")
plt.title("Learning Curve")
plt.grid(True)
plt.yscale("log")
plt.show()

```

What PyTorch is actually doing

For an M -tap adaptive FIR filter,

$$y[n] = \sum_{k=0}^{M-1} w_k[n] x[n-k].$$

Define the error as

$$e[n] = d[n] - y[n].$$

We minimize

$$J[n] = e^2[n].$$

In the code, these three equations correspond almost literally to:

```
y = torch.dot(w, x_vec)
```

```
e = d_t[n] - y
```

```
loss = e**2
```

Then

```
loss.backward()
```

asks PyTorch to calculate

$$\frac{\partial J}{\partial w_k}.$$

For this simple filter, PyTorch will discover through autograd that

$$\frac{\partial J}{\partial w_k} = -2e[n]x[n-k].$$

The SGD optimizer then performs

$$w_k[n+1] = w_k[n] - \mu \frac{\partial J}{\partial w_k}.$$

Therefore,

$$w_k[n+1] = w_k[n] + 2\mu e[n]x[n-k].$$

That is essentially the **LMS adaptive filter update**. The factor of 2 can simply be absorbed into the learning rate.

So there is a nice teaching point here:

An LMS adaptive filter can be viewed as stochastic gradient descent on a one-sample squared-error loss.

An especially useful modification for a DSP/ML class

I would actually show the students **both updates side-by-side**.

Traditional LMS:

```
with torch.no_grad():
    w += mu * e * x_vec
```

versus the PyTorch/ML version:

```
loss = e**2
```

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

The first says, "**We derived the gradient analytically.**"

The second says, "**Let PyTorch construct the computational graph and derive the gradient for us.**"

For a DSP-to-machine-learning course, this is a particularly useful bridge because the exact same `loss.backward()` / `optimizer.step()` pattern later trains a neural network; only the model between x and y becomes much more complicated.

So what is pytorch actually training on?

ChatGPT Medium:

PyTorch is training only the **adaptive filter coefficients** $w_0, w_1, \dots, w_{M-1}$.

In the example, this line makes the FIR coefficients trainable:

```
w = torch.nn.Parameter(torch.zeros(M))
```

So the "model" is just

$$y[n] = \mathbf{w}^T \mathbf{x}[n].$$

At each time step, PyTorch is given two things: the current input vector

$$\mathbf{x}[n] = [x[n], x[n-1], \dots, x[n-M+1]]^T$$

and the desired output $d[n]$.

It computes the filter output

$$y[n] = \mathbf{w}^T \mathbf{x}[n],$$

then the error

$$e[n] = d[n] - y[n],$$

and then the loss

$$J[n] = e^2[n].$$

So when you ask, "What is PyTorch training on?", there are two useful meanings.

The **training data** is the sequence of input/desired-output pairs:

$$(\mathbf{x}[n], d[n]).$$

The **trainable quantities** are the coefficients:

$$\mathbf{w}.$$

For example, suppose at one instant you have

$$\begin{aligned} \mathbf{x_vec} &= [1.2, -0.5, 0.8, 0.1] \\ d[n] &= 0.93 \end{aligned}$$

and the current coefficients are

$$\mathbf{w} = [0.10, 0.10, 0.10, 0.10].$$

The current output is

$$y[n] = 0.10(1.2) + 0.10(-0.5) + 0.10(0.8) + 0.10(0.1) = 0.16.$$

So the error is

$$e[n] = 0.93 - 0.16 = 0.77.$$

PyTorch then asks:

How should I change the four numbers in $\mathbf{w}$ so that this output would have been closer to 0.93?

`loss.backward()` computes that direction, and `optimizer.step()` moves the coefficients slightly in that direction.

The important distinction is that PyTorch is **not training the input signal** and it is **not training the desired signal**. Those are fixed observations. It is adjusting only:

$\mathbf{w}$

because that's the only object declared as a Parameter.

This makes the adaptive-filter example unusually simple compared with a neural network. In a neural network you might have thousands or millions of trainable weights. Here you have only four:

$$w_0, \quad w_1, \quad w_2, \quad w_3.$$

In system-identification language, the situation is:

$$x[n] \rightarrow \boxed{\text{unknown system } h} \rightarrow d[n]$$

and simultaneously

$$x[n] \rightarrow \boxed{\text{adaptive filter } w} \rightarrow y[n].$$

The adaptation tries to make

$$y[n] \approx d[n].$$

Therefore, after enough samples,

$$\mathbf{w} \approx \mathbf{h}.$$

So in the code I gave you, PyTorch is essentially being trained on **examples of what the unknown system does**. It sees an input $x[n]$ and the resulting desired output $d[n]$, and it adjusts its FIR coefficients until its own input-output behavior mimics the unknown system.

That's exactly why adaptive system identification is such a nice introductory PyTorch example: **the adaptive FIR filter is really just a one-layer linear neural network with no bias**.

DSP traditional LMS

Google:

In Digital Signal Processing (DSP), the Traditional LMS (Least Mean Squares) algorithm is a fundamental adaptive filtering technique used to mimic a desired signal by modifying filter coefficients in real time. Here is a breakdown of how the traditional LMS algorithm works, its core mathematics, and its limitations. -----

The Core Concept

The LMS algorithm is based on the stochastic gradient descent method. It iteratively adjusts the weights (coefficients) of a Finite Impulse Response (FIR) filter to minimize the Mean Squared Error (MSE) between the filter's output and a desired target signal.

Input $x(n)$ -----> [Adaptive FIR Filter] -----> Output $y(n)$

|
v

|
v

Desired $d(n)$ -----> (+) <--- [Subtract] -----+ | v Error $e(n)$ -----> (Updates filter weights)

The 3-Step Mathematical Loop

For every incoming time sample n , the traditional LMS algorithm executes three strict sequential steps:

1. Filter Output (Filtering)

The filter calculates its current output $y(n)$ by taking the dot product of the input vector $\mathbf{x}(n)$ and the current weight vector $\mathbf{w}(n)$:

$$y(n) = \mathbf{w}^T(n)\mathbf{x}(n) = \sum_{i=0}^{N-1} w_i(n)x(n-i)$$

(where N is the number of filter taps)

2. Error Estimation

The algorithm calculates the estimation error $e(n)$ by comparing the desired signal $d(n)$ with the actual filter output:

$$e(n) = d(n) - y(n)$$

3. Weight Update (Adaptation)

The filter weights are updated for the next sample $(n+1)$ by moving in the negative direction of the gradient of the squared error:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)$$

- μ (Step-Size / Convergence Factor): A small positive constant that controls how fast and how stably the filter learns.

Convergence and Stability (μ)

The selection of the step-size μ is critical to making traditional LMS work:

- If μ is too large: The algorithm becomes unstable, and the filter weights will diverge (blow up).
- If μ is too small: The filter learns incredibly slowly, making it useless for fast-changing tracking environments.
- Condition for Stability: To guarantee convergence, μ must satisfy:

$$0 < \mu < \frac{1}{\lambda_{\max}}$$

(where $\lambda_{\max}$ is the largest eigenvalue of the input signal's autocorrelation matrix R)

Limitations of Traditional LMS

While highly popular due to its simplicity, traditional LMS has distinct drawbacks compared to modern variations:

- Sensitivity to Input Power: If the power of the input signal $x(n)$ suddenly increases, the effective step size increases, which can cause the filter to diverge. (This led to the invention of NLMS—Normalized LMS).
 - Slow Convergence with Colored Noise: If the input signal is highly correlated (like speech or music, rather than white noise), LMS converges very slowly.
 - Fixed Step-Size Dilemma: You cannot have both fast tracking and low steady-state error. You must choose one.
-

If you are looking to implement or analyze this further, let me know:

- Are you writing this in a specific language? (e.g., MATLAB, Python, C)
- What is your specific application? (e.g., acoustic echo cancellation, system identification, noise removal)
- Do you need to compare it to NLMS or RLS algorithms?

I can provide code snippets or tailored performance trade-offs based on what you are building.

I'm writing this in Python. Can you give me a small numerical example?

Google:

Here is a complete Python script that implements a 2-tap LMS adaptive filter alongside a step-by-step trace of the math for the first two samples.

The Python Code

This script uses a target system with known weights $[0.5, -0.3]$. The LMS algorithm starts with zero weights $[0.0, 0.0]$ and attempts to learn the target values sample-by-sample.

```
import numpy as np
```

1. Setup the true target system we want to identify w_{true}
`= np.array([0.5, -0.3])`

2. Input signal $x(n)$ and initial filter setup
`x = np.array([1.0, 2.0, 0.5, -1.0, 1.0])`
`w = np.zeros(2) # Initial weights [0.0, 0.0]`
`mu = 0.1 # Step-size parameter`

```
print(f"Initial weights: {w}\n" + "-"*45)
```

3. The LMS Loop for n in range(len(x)):

```
# Form the input vector x_vec = [x(n), x(n-1)]
x0 = x[n]
x1 = x[n-1] if n > 0 else 0.0
x_vec = np.array([x0, x1])

# Generate the ideal desired signal d(n) using the true weights
d = np.dot(w_true, x_vec)

# Step 1: Compute filter output y(n)
y = np.dot(w, x_vec)

# Step 2: Compute error e(n)
e = d - y

# Step 3: Update weights
w = w + 2 * mu * e * x_vec

# Print the step-by-step evolution
print(f"Sample n={n}")
print(f"  Input Vector x(n) : {x_vec}")
print(f"  Desired d(n)       : {d:6.2f} | Output y(n) : {y:6.2f}")
print(f"  Error e(n)         : {e:6.2f}")
print(f"  Updated Weights    : {w}")
print("-"*45)
```

Numerical Walkthrough (The Math)

Here is exactly how the numbers change during the first two loops:

Sample n = 0

- Inputs: Current input $x(0) = 1.0$, past input $x(-1) = 0.0$. Vector $\mathbf{x}(0) = [1.0, 0.0]$.
- Target Signal: $d(0) = (0.5 \times 1.0) + (-0.3 \times 0.0) = \mathbf{0.5}$
- Step 1 (Output): $y(0) = (0.0 \times 1.0) + (0.0 \times 0.0) = \mathbf{0.0}$
- Step 2 (Error): $e(0) = 0.5 - 0.0 = \mathbf{0.5}$
- Step 3 (Update):

$$\begin{aligned}\mathbf{w}(1) &= [0.0, 0.0] + 2 \times 0.1 \times 0.5 \times [1.0, 0.0] \\ \mathbf{w}(1) &= [0.0, 0.0] + 0.1 \times [1.0, 0.0] = [\mathbf{0.1}, \mathbf{0.0}]\end{aligned}$$

Sample n = 1

- Inputs: Current input $x(1) = 2.0$, past input $x(0) = 1.0$. Vector $\mathbf{x}(1) = [2.0, 1.0]$.
- Target Signal: $d(1) = (0.5 \times 2.0) + (-0.3 \times 1.0) = \mathbf{0.7}$
- Step 1 (Output): Using weights from previous step $[0.1, 0.0]$:
 $y(1) = (0.1 \times 2.0) + (0.0 \times 1.0) = \mathbf{0.2}$
- Step 2 (Error): $e(1) = 0.7 - 0.2 = \mathbf{0.5}$
- Step 3 (Update):

$$\begin{aligned}\mathbf{w}(2) &= [0.1, 0.0] + 2 \times 0.1 \times 0.5 \times [2.0, 1.0] \\ \mathbf{w}(2) &= [0.1, 0.0] + 0.1 \times [2.0, 1.0] = [\mathbf{0.3}, \mathbf{0.1}]\end{aligned}$$

As you can see, by sample $n=4$, the weights progress to $[0.354, -0.271]$, steadily marching closer to the true targets of $[0.5, -0.3]$. Would you like to scale this up into an interactive simulation using real audio data or synthetic white noise to see how it acts in a realistic streaming scenario?

```
import numpy as np
```

Let's create a minimal system identification example#

Target system: 2-tap filter with weights $[0.5, -0.3]$ # Input x:
a simple sequence

```
np.random.seed(42)x = np.array([1.0, 2.0, 0.5, -1.0, 1.0])w_true = np.array([0.5, -0.3])
```

Generate desired signal $d(n) = w_true[0]x(n) + w_true[1]x(n-1)$ # For $n=0$, $x(-1) = 0$
 $d = np.zeros(len(x))$ for n in range(len(x)):

```
x_n = x[n]
x_n_minus_1 = x[n-1] if n > 0 else 0.0
d[n] = w_true[0]*x_n + w_true[1]*x_n_minus_1
```

LMS Setup

`w = np.zeros(2) # Initial weights`
`mu = 0.1 # Step size`

```
history = []
for n in range(len(x)): # Form input vector x_vec = [x(n), x(n-1)]
    x0 = x[n]
    x1 = x[n-1]
    if n > 0 else 0.0
    x_vec = np.array([x0, x1])

    # 1. Filter output
    y = np.dot(w, x_vec)

    # 2. Error
    e = d[n] - y

    # Record before update
    w_old = w.copy()

    # 3. Weight update
    w = w + 2 * mu * e * x_vec

    history.append({
        'n': n,
        'x_vec': x_vec.tolist(),
        'd': d[n],
        'y': y,
        'e': e,
        'w_new': w.tolist()
    })

import pprint
pprint.pprint(history)
```

Yes, I would like to scale this up for real audio data

Google:

Here is your file: This production-ready Python script expands the 2-tap numerical framework into a complete, streaming LMS Adaptive Noise Cancellation Pipeline tailored for handling raw time-domain audio data.

How this Audio Pipeline Works

In real audio streaming (like in noise-canceling headphones or speakerphone echo cancellation), you typically have two data streams:

1. The Primary Microphone Signal ($d[n]$): The desired audio voice heavily corrupted by background environment noise.

2. The Reference Noise Signal ($x[n]$): A secondary mic placed near the noise source (e.g., outside the headphone cup) that captures the raw ambient noise before it alters and mixes with your voice.

The LMS filter processes the reference noise stream $x[n]$, shapes it to mimic the physical acoustics of the environment, and dynamically subtracts it from the primary microphone signal—leaving behind a purified, noise-canceled audio stream.

Python Implementation Setup

The generated code includes a built-in generator that simulates an environmental channel and outputs standard 16-bit PCM Mono .wav files using Python's native wave library (eliminating any heavy external dependencies like scipy or soundfile).

```
import numpy as np
import wave

def save_wav(filename, data, fs=8000): """Normalizes and
saves a numpy float array as a standard 16-bit mono WAV file."""
    max_val = np.max(np.abs(data))
    if max_val > 0: data = (data / max_val) * 32767
    data = data.astype(np.int16)

    with wave.open(filename, 'w') as f:
        f.setnchannels(1) # Mono
        f.setsampwidth(2) # 16-bit PCM
        f.setframerate(fs)
        f.writeframes(data.tobytes())
    print(f"Saved: {filename}")

def run_lms_noise_cancellation():
    fs = 8000
    duration = 5.0 # 5 seconds of audio
    t = np.linspace(0, duration, int(fs * duration), endpoint=False)

    # 1. Target Clean Voice/Audio Tone
    clean_audio = np.sin(2 * np.pi * 350 * t) + 0.5 * np.sin(2 * np.pi * 700 * t)

    # 2. Reference Noise Source x(n) (e.g., drone hum/background static)
    noise_ref = np.random.normal(0, 0.4, len(t))

    # 3. Environmental Path Simulation (The unknown physical distortion)
    true_acoustic_path = np.array([0.7, -0.4, 0.2, -0.1])
    num_taps = len(true_acoustic_path)

    noise_at_mic = np.zeros_like(noise_ref)
    for n in range(num_taps, len(noise_ref)):
        x_vec = noise_ref[n-num_taps:n][::-1]
        noise_at_mic[n] = np.dot(true_acoustic_path, x_vec)

    # 4. Microphone Input (Desired Signal d(n) = Voice + Distorted Noise)
    mic_signal = clean_audio + noise_at_mic

    # 5. Initialize LMS Adaptive Filter
```

```

w = np.zeros(num_taps) # Start with zeroed weight beliefs
mu = 0.015             # Learning rate step size
filtered_output = np.zeros_like(mic_signal)

# 6. Real-Time Streaming LMS Loop
for n in range(num_taps, len(mic_signal)):
    # Construct the historical buffer window of noise samples
    x_vec = noise_ref[n-num_taps:n][::-1]

    # Step 1: Predict the structural shape of the incoming noise distortion
    predicted_noise = np.dot(w, x_vec)

    # Step 2: Cancel noise out of the microphone input to get the clean
estimate
    #  $e(n) = d(n) - y(n)$ 
    error_clean = mic_signal[n] - predicted_noise
    filtered_output[n] = error_clean

    # Step 3: Shift the weights along the negative gradient vector
    w = w + 2 * mu * error_clean * x_vec

print(f"Final Trained LMS Weights: {w}")
print(f"True Environmental Path: {true_acoustic_path}")

# Export audio assets to files
save_wav("microphone_noisy.wav", mic_signal, fs)
save_wav("lms_cleaned_output.wav", filtered_output, fs)
save_wav("original_pure_clean.wav", clean_audio, fs)

if name == "main": run_lms_noise_cancellation()

```

What to Expect When You Run It

- **Convergence Behavior:** If you inspect the `lms_cleaned_output.wav` file, you will notice a tiny burst of noise right at the beginning (0.0 to 0.2 seconds). This is the exact moment the LMS algorithm takes to actively "learn" the environmental filter weights.
- **Noise Suppression:** After those fractions of a second, the background static drops away dramatically, leaving a distinct, clean audio signal.

Would you like to extend this script to read an actual .wav file from your hard drive, or should we modify it to handle Normalized LMS (NLMS) to prevent stability issues when handling erratic volume spikes?

Yes, modify it to read an actual .wav file from my hard drive

Google:

Here is your file: This modified, standalone Python script expands the streaming framework to read and process actual audio files directly from your hard drive.

Key Modifications for Hard Drive Audio

- **Native Audio Parsing:** It utilizes Python's built-in `wave` module, completely eliminating the need for bulky third-party libraries like `scipy.io.wavfile` or `soundfile`.
- **Bit-Depth Normalization:** Real audio files are stored as integers (typically 16-bit PCM). The script reads the binary frames, scales them down to floating-point values between `[-1.0, 1.0]` for stable LMS convergence, and automatically handles multi-channel files by extracting a single mono channel.
- **Safe Rescaling:** The output filtered array is safely clipped and re-scaled back to standard 16-bit integer parameters upon saving to prevent audio clipping distortion.

The Python WAV Processor Script

```
import numpy as np
import wave
import sys

def read_wav(filename):
    """Reads a standard 16-bit mono WAV file and returns normalized float data and sample rate."""
    try:
        with wave.open(filename, 'rb') as f:
            channels = f.getnchannels()
            sample_width = f.getsampwidth()
            fs = f.getframerate()
            n_frames = f.getnframes()

            if sample_width != 2:
                raise ValueError(f"Unsupported bit depth ({sample_width * 8}-bit). Please use a standard 16-bit PCM WAV file.")

            # Read raw binary frames
            raw_data = f.readframes(n_frames)
            data_int = np.frombuffer(raw_data, dtype=np.int16)

            # Convert to mono if the file is stereo
            if channels > 1:
                data_int = data_int.reshape(-1, channels)[: , 0]

            # Normalize to [-1.0, 1.0] float range for LMS stability
            data_float = data_int.astype(np.float32) / 32768.0
            return data_float, fs
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found. Please update the path variables.")
        sys.exit(1)

def save_wav(filename, data, fs):
    """Normalizes and saves a numpy float array as a standard 16-bit mono WAV file."""
    # Protect against extreme gradient explosion spikes
```

```

before serialization data = np.clip(data, -1.0, 1.0) data_int = (data *
32767.0).astype(np.int16)

with wave.open(filename, 'w') as f:
    f.setnchannels(1)
    f.setsampwidth(2)
    f.setframerate(fs)
    f.writeframes(data_int.tobytes())
print(f"Successfully saved: {filename}")

def run_wav_file_lms(input_wav_path, output_wav_path, num_taps=32, mu=0.01):
    """Applies LMS adaptive noise cancellation to an actual WAV file.""" print(f"Loading input
file: {input_wav_path}...") mic_signal, fs = read_wav(input_wav_path) N = len(mic_signal)
print(f"Loaded {N} samples at a sample rate of {fs}Hz (~{N/fs:.2f} seconds).")

    # Generate a reference noise source x(n)
    np.random.seed(42)
    reference_noise = np.random.normal(0, 0.05, N)

    # Simulate an environmental path leaking that noise into your file
    acoustic_leakage = np.array([0.6, -0.3, 0.1])
    noisy_mic_signal = np.copy(mic_signal)

    for n in range(len(acoustic_leakage), N):
        x_buffer = reference_noise[n-len(acoustic_leakage):n][::-1]
        noisy_mic_signal[n] += np.dot(acoustic_leakage, x_buffer)

    # Save the corrupted version for a side-by-side comparison
    save_wav("noisy_input_preview.wav", noisy_mic_signal, fs)

    # Initialize LMS Adaptive Filter
    w = np.zeros(num_taps)
    filtered_output = np.zeros(N)

    print(f"Running LMS Adaptive Filter ({num_taps} Taps, Step-size  $\mu$  =
{mu})...")

    # Streaming LMS Loop
    for n in range(num_taps, N):
        x_vec = reference_noise[n-num_taps:n][::-1]

        # Step 1: Predict the shape of the noise leak
        predicted_noise = np.dot(w, x_vec)

        # Step 2: Subtract it from the primary mic signal
        error_clean = noisy_mic_signal[n] - predicted_noise
        filtered_output[n] = error_clean

        # Step 3: Update filter weights

```

```

w = w + 2 * mu * error_clean * x_vec

print("Processing complete!")
save_wav(output_wav_path, filtered_output, fs)

if name == "main": #  CONFIGURATION: Point these to your hard drive file locations
INPUT_FILE = "your_voice_file.wav" OUTPUT_FILE = "lms_cleaned_output.wav"

# Safe fallback handler: Creates a placeholder if you run it before copying
your file over
try:
    with wave.open(INPUT_FILE, 'r'): pass
except FileNotFoundError:
    print(f"Generating a placeholder '{INPUT_FILE}' (440Hz Sine Tone) since
it wasn't found...")
    t = np.linspace(0, 2, 8000 * 2, endpoint=False)
    dummy_voice = 0.5 * np.sin(2 * np.pi * 440 * t)
    save_wav(INPUT_FILE, dummy_voice, 8000)

# Run the adaptive pipeline
run_wav_file_lms(INPUT_FILE, OUTPUT_FILE, num_taps=16, mu=0.02)

```

If you run this with your own files, let me know:

- Is your input file already noisy, or do you need to read a second reference noise file captured from a real dual-mic setup?
- Are you running into stability issues (audio clipping/harsh static) when updating the weights?

I can show you how to swap this out for Normalized LMS (NLMS) if the signal levels vary wildly.

Yes, NLMS would be good

Google:

Here is your file:

- Generated file: nlms_wav_processor.py

Upgrading to NLMS (Normalized Least Mean Squares) is the correct choice for real audio. Because voice signals naturally have massive power swings (loud words vs. silent gaps), a fixed step-size μ in traditional LMS easily causes filter divergence during loud sounds, or stops learning during quiet ones.

What changes mathematically?

Traditional LMS updates weights using a fixed step-size μ :

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)$$

NLMS solves the volume-spike problem by normalizing the step size by the total energy current residing inside the filter's buffer window ($\|\mathbf{x}(n)\|^2$):

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\mu_{nlms}}{\|\mathbf{x}(n)\|^2 + \epsilon} e(n)\mathbf{x}(n)$$

- μ_{nlms} (Normalized Step Size): A scaled learning rate bounded between 0 and 2. Usually set between 0.1 and 0.5 for stable tracking.
 - ϵ (Epsilon): A tiny stabilization constant (e.g., 1e-6) to prevent a catastrophic division-by-zero error when the audio goes completely silent.
-

The Python NLMS WAV Processor

This updated script modifies the real-time update loop to perform normalization on every step.

```
import numpy as np
import wave
import sys

def read_wav(filename):
    """Reads a standard 16-bit mono WAV file and returns normalized float data and sample rate."""
    try:
        with wave.open(filename, 'rb') as f:
            channels = f.getnchannels()
            sample_width = f.getsampwidth()
            fs = f.getframerate()
            n_frames = f.getnframes()
```

```
        if sample_width != 2:
            raise ValueError(f"Unsupported bit depth ({sample_width * 8}-bit). Please use a standard 16-bit PCM WAV file.")
```

```
        raw_data = f.readframes(n_frames)
        data_int = np.frombuffer(raw_data, dtype=np.int16)
```

```
        if channels > 1:
            data_int = data_int.reshape(-1, channels)[: , 0]
```

```
        data_float = data_int.astype(np.float32) / 32768.0
        return data_float, fs
```

```
except FileNotFoundError:
    print(f"Error: The file '{filename}' was not found.")
    sys.exit(1)
```

```
def save_wav(filename, data, fs):
    """Normalizes and saves a numpy float array as a standard 16-bit mono WAV file."""
    data = np.clip(data, -1.0, 1.0)
    data_int = (data * 32767.0).astype(np.int16)
```

```

with wave.open(filename, 'w') as f:
    f.setnchannels(1)
    f.setsampwidth(2)
    f.setframerate(fs)
    f.writeframes(data_int.tobytes())
print(f"Successfully saved: {filename}")

def run_wav_file_nlms(input_wav_path, output_wav_path, num_taps=32, mu_nlms=0.2):
    """Applies Normalized LMS (NLMS) adaptive noise cancellation to a WAV file."""
    print(f"Loading input file: {input_wav_path}...")
    mic_signal, fs = read_wav(input_wav_path)
    N = len(mic_signal)

    # 1. Generate an unpredictable reference noise source x(n)
    np.random.seed(42)
    reference_noise = np.random.normal(0, 0.1, N)

    # 2. Simulate acoustic path leaking that noise into the microphone signal
    acoustic_leakage = np.array([0.5, -0.25, 0.12])
    noisy_mic_signal = np.copy(mic_signal)
    for n in range(len(acoustic_leakage), N):
        x_buffer = reference_noise[n-len(acoustic_leakage):n][::-1]
        noisy_mic_signal[n] += np.dot(acoustic_leakage, x_buffer)

    save_wav("noisy_input_preview.wav", noisy_mic_signal, fs)

    # 3. Initialize NLMS Variables
    w = np.zeros(num_taps)
    filtered_output = np.zeros(N)
    eps = 1e-6 # Tiny epsilon constant to prevent division by zero during
    silence

    print(f"Running NLMS Filter ({num_taps} Taps, Normalized Step-size  $\mu_{nlms} =$ 
    {mu_nlms})...")

    # 4. Processing Loop
    for n in range(num_taps, N):
        x_vec = reference_noise[n-num_taps:n][::-1]

        # Step 1: Predict noise leak
        predicted_noise = np.dot(w, x_vec)

        # Step 2: Error subtraction (Filter cancellation output)
        error_clean = noisy_mic_signal[n] - predicted_noise
        filtered_output[n] = error_clean

        # Step 3: Compute signal power in current buffer window
        norm_factor = np.dot(x_vec, x_vec) + eps

        # Step 4: Update weights with Normalized Step Size

```

```

w = w + (mu_nlms / norm_factor) * error_clean * x_vec

print("NLMS Processing complete!")
save_wav(output_wav_path, filtered_output, fs)

if name == "main": INPUT_FILE = "your_voice_file.wav" OUTPUT_FILE =
"nlms_cleaned_output.wav"

# Generate placeholder file if missing
try:
    with wave.open(INPUT_FILE, 'r'): pass
except FileNotFoundError:
    print(f"Generating placeholder '{INPUT_FILE}'...")
    t = np.linspace(0, 3, 8000 * 3, endpoint=False)
    dummy_voice = 0.4 * np.sin(2 * np.pi * 300 * t) # 300Hz base tone
    save_wav(INPUT_FILE, dummy_voice, 8000)

# Run the adaptive pipeline using normalized LMS
run_wav_file_nlms(INPUT_FILE, OUTPUT_FILE, num_taps=32, mu_nlms=0.25)

```

If you want to tune this script specifically to your audio file, tell me:

- What kind of noise is contaminating your audio? (e.g., constant hum, wind, erratic background chatter?)
- Do you know the sample rate of your audio file?

Knowing this will help determine the optimal number of filter taps needed to cleanly isolate your audio.

If I have a cost function $C(\mathbf{h}) = \sum_{n=0}^N [y[n] - \sum_{k=0}^{M-1} h[k]x[n-k]]^2$, what is the partial derivative with respect to h ?

ChatGPT Medium:

Your cost function is

$$C(\mathbf{h}) = \sum_{n=0}^N \left[y[n] - \sum_{k=0}^{M-1} h[k]x[n-k] \right]^2.$$

Let

$$e[n] = y[n] - \sum_{k=0}^{M-1} h[k]x[n-k].$$

Then

$$C(\mathbf{h}) = \sum_{n=0}^N e^2[n].$$

For a particular coefficient $h[m]$,

$$\frac{\partial C}{\partial h[m]} = \sum_{n=0}^N 2 e[n] \frac{\partial e[n]}{\partial h[m]}.$$

Since

$$e[n] = y[n] - \sum_{k=0}^{M-1} h[k]x[n-k],$$

we have

$$\frac{\partial e[n]}{\partial h[m]} = -x[n-m].$$

Therefore,

$$\boxed{\frac{\partial C}{\partial h[m]} = -2 \sum_{n=0}^N e[n]x[n-m]}$$

or, substituting the error explicitly,

$$\boxed{\frac{\partial C}{\partial h[m]} = -2 \sum_{n=0}^N \left[y[n] - \sum_{k=0}^{M-1} h[k]x[n-k] \right] x[n-m]}$$

for $m = 0, 1, \dots, M-1$.

In vector form, if

$$\mathbf{x}_n = \begin{bmatrix} x[n] \\ x[n-1] \\ \vdots \\ x[n-M+1] \end{bmatrix},$$

and

$$\hat{y}[n] = \mathbf{h}^T \mathbf{x}_n,$$

then

$$C(\mathbf{h}) = \sum_{n=0}^N (y[n] - \mathbf{h}^T \mathbf{x}_n)^2,$$

and the gradient is

$$\nabla_{\mathbf{h}} C = -2 \sum_{n=0}^N \mathbf{x}_n (y[n] - \mathbf{h}^T \mathbf{x}_n)$$

which is exactly the gradient that an LMS-style or PyTorch gradient-descent update uses.

Now this is used to update $\mathbf{h}$ to $\mathbf{h}(t+1)$ by subtracting μ times $\nabla_{\mathbf{h}} C$ from $\mathbf{h}(t)$

ChatGPT Medium:

Exactly. With gradient descent, the coefficient vector is updated as

$$\mathbf{h}^{(t+1)} = \mathbf{h}^{(t)} - \mu \nabla_{\mathbf{h}} C.$$

Using the gradient we derived,

$$\nabla_{\mathbf{h}} C = -2 \sum_{n=0}^N \mathbf{x}_n (y[n] - \mathbf{h}^T \mathbf{x}_n),$$

so the update becomes

$$\mathbf{h}^{(t+1)} = \mathbf{h}^{(t)} + 2\mu \sum_{n=0}^N \mathbf{x}_n (y[n] - \mathbf{h}^{(t)T} \mathbf{x}_n).$$

For an individual coefficient $h[m]$,

$$h^{(t+1)}[m] = h^{(t)}[m] + 2\mu \sum_{n=0}^N \left[y[n] - \sum_{k=0}^{M-1} h^{(t)}[k] x[n-k] \right] x[n-m].$$

A useful point is that this version uses the **entire dataset** to compute one gradient update. That is batch gradient descent.

For an adaptive filter such as LMS, we usually use just the current sample n as an instantaneous estimate of the gradient:

$$\nabla_{\mathbf{h}} C_n = -2e[n]\mathbf{x}_n.$$

Then

$$\mathbf{h}[n+1] = \mathbf{h}[n] + 2\mu e[n]\mathbf{x}_n$$

where

$$e[n] = y[n] - \mathbf{h}^T[n]\mathbf{x}_n.$$

Often the factor of 2 is absorbed into the step size, giving the familiar LMS update

$$\boxed{\mathbf{h}[n+1] = \mathbf{h}[n] + \mu e[n]\mathbf{x}_n}$$

So the main distinction is:

- **Gradient descent:** compute the error over many or all samples, then update $\mathbf{h}$.
- **LMS / stochastic gradient descent:** use one new sample, make one update immediately.

That connection is exactly why PyTorch's SGD optimizer maps so naturally onto adaptive filtering.
